Generative KI-Modelle und ihre mathematischen Grundlagen

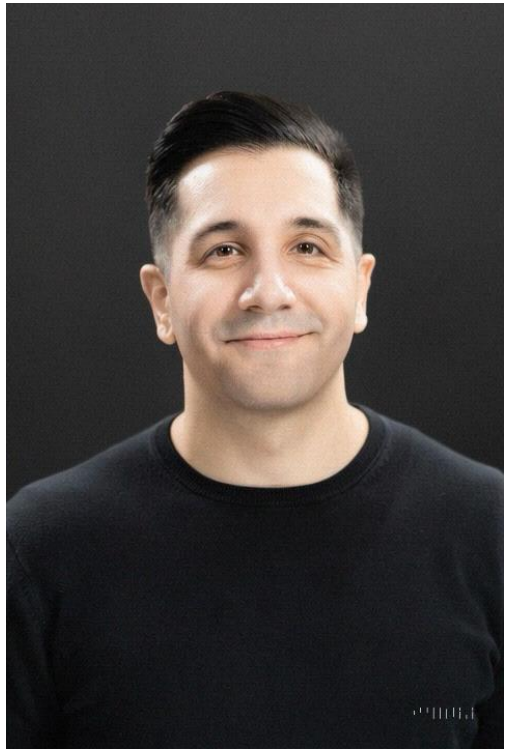
Orhan Konak
Hochschule Bremen
24.04.2026

Motivation

Wer von euch nutzt ein oder mehrere der folgenden Tools (Bitte Hand heben)?

- ChatGPT (oder anderes Sprachmodell)
- DALL·E / Midjourney
- Sora / Veo
- Code-Generatoren

ich, orhan konak, während einer probelehrveranstaltung an der hochschule bremen in einem seminarraum, vor studierenden und der berufungskommission, zum thema "generative ki-modelle und ihre mathematischen grundlagen" mit fokus auf diffusion models



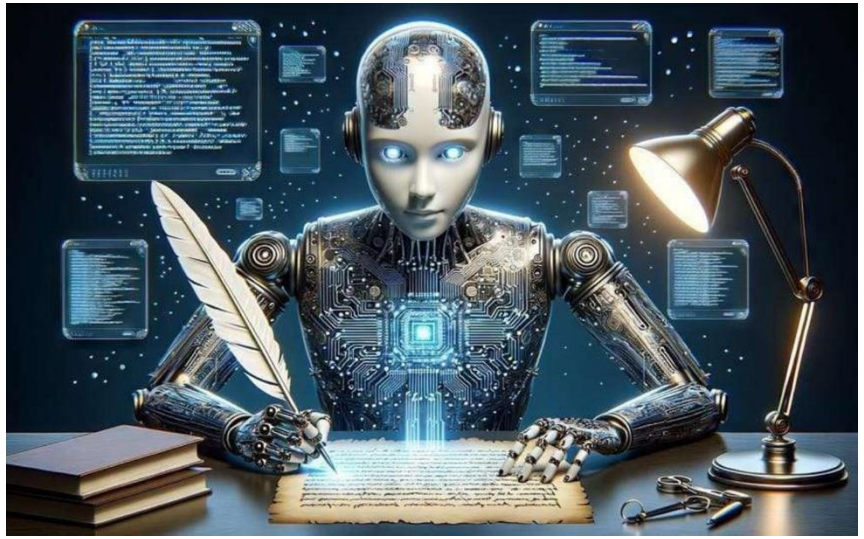
```
import os
import math
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import imageio.v2 as imageio
# ==========================================================================
# 1) Einstellungen
# ==========================================================================
IMAGE_PATH = "Orhan_MU.jpg" # Pfad zu deinem Bild
OUTPUT_DIR = "diffusion_forward_output"
T = 1000
TARGET_SIZE = (256, 256)
# Schritte, die als Einzelbilder/PDF gespeichert werden
VIS_STEPS = [0, 20, 50, 100, 150, 250, 400, 600, 800, 999]
# ...
```

Warum sprechen alle über Generative KI?

- ChatGPT schreibt Texte → nimmt uns Arbeit ab
- DALL·E / Midjourney erzeugt Bilder → Datenschutz
- KI generiert Code, Musik, medizinische Daten → neue Möglichkeiten

→ Systeme erzeugen *neue Inhalte*, nicht nur Vorhersagen

Aber: *Wie kann ein Modell neue Daten erzeugen, die es nie zuvor gesehen hat?*



https://www.kofana.com/en/trends/reports-and-blog/digital-transformation-in-business-with-generative-ai

Agenda

- Was bedeutet „Generieren“ in der KI?
- Wie lassen sich Daten mathematisch beschreiben?
- Wie lernen Modelle solche Verteilungen?
- Beispiel: Generierung mit Diffusion Models
- Zusammenfassung

Agenda

- **Was bedeutet „Generieren“ in der KI?**
- Wie lassen sich Daten mathematisch beschreiben?
- Wie lernen Modelle solche Verteilungen?
- Beispiel: Generierung mit Diffusion Models
- Zusammenfassung

Diskriminative vs. Generative Modelle

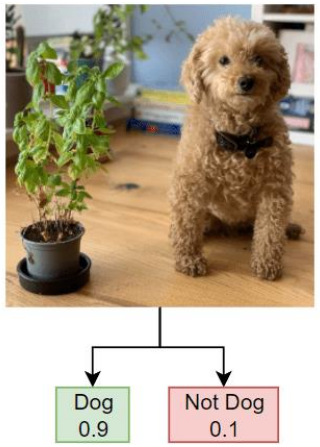
Wie wahrscheinlich ist y, wenn ich x schon kenne?

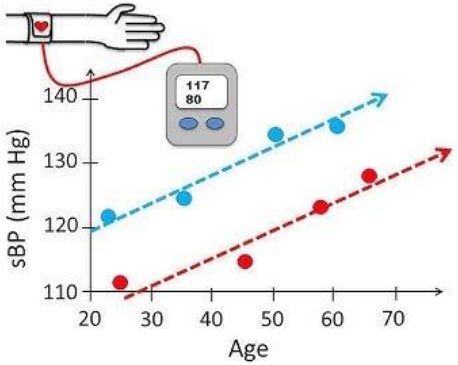
Diskriminative Modelle

- Lernen: $p(y \mid x)$
- Klassifikation / Regression
- Beispiel: Spam-Erkennung

Generative Modelle

- Lernen: $p(x)$ oder $p(x, y)$
- Ziel: Daten erzeugen
- Beispiel: Bildgenerierung





[1] https://de.mathworks.com/help/deeplearning/ug/multilabel-image-classification-using-deep-learning.html
[2] https://www.youtube.com/watch?v=AP_K7SaKkIE

Generative Modelle lernen die *Struktur der Daten selbst*

Zentrale Fragestellung

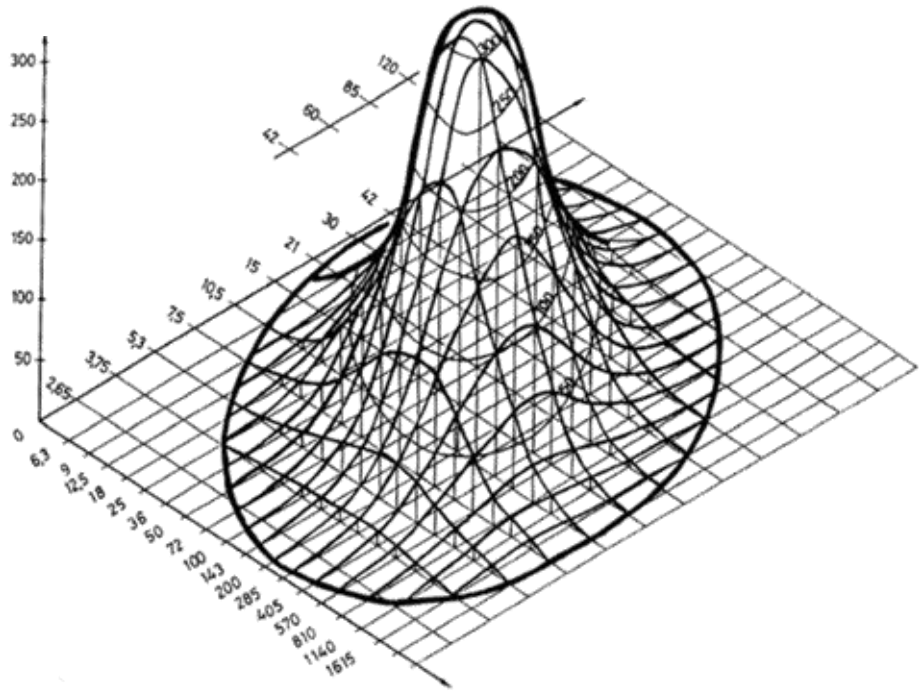
Wie kann ein Modell Daten erzeugen, die es nie zuvor gesehen hat?

- Lernen wir Muster?
- Lernen wir Regeln?
- Oder etwas anderes?

Bottom Line:

- Wir lernen eine ***Wahrscheinlichkeitsverteilung***

$p(x)$



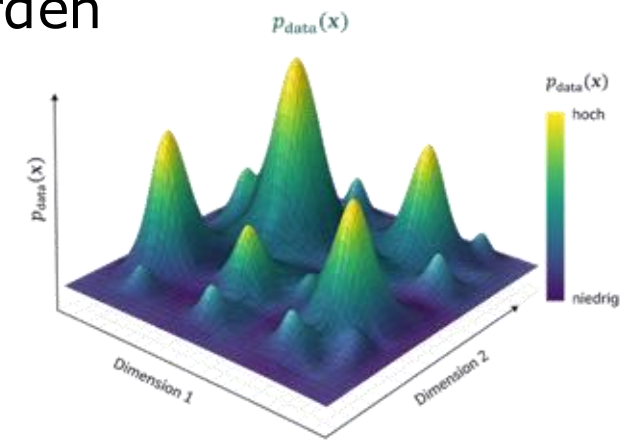
Agenda

- Was bedeutet „Generieren“ in der KI?
- **Wie lassen sich Daten mathematisch beschreiben?**
- Wie lernen Modelle solche Verteilungen?
- Beispiel: Generierung mit Diffusion Models
- Zusammenfassung

Daten als Wahrscheinlichkeitsverteilung

Inhalt:

- Daten sind nicht zufällig
- Sie folgen einer Struktur
- Diese Struktur kann als Verteilung beschrieben werden

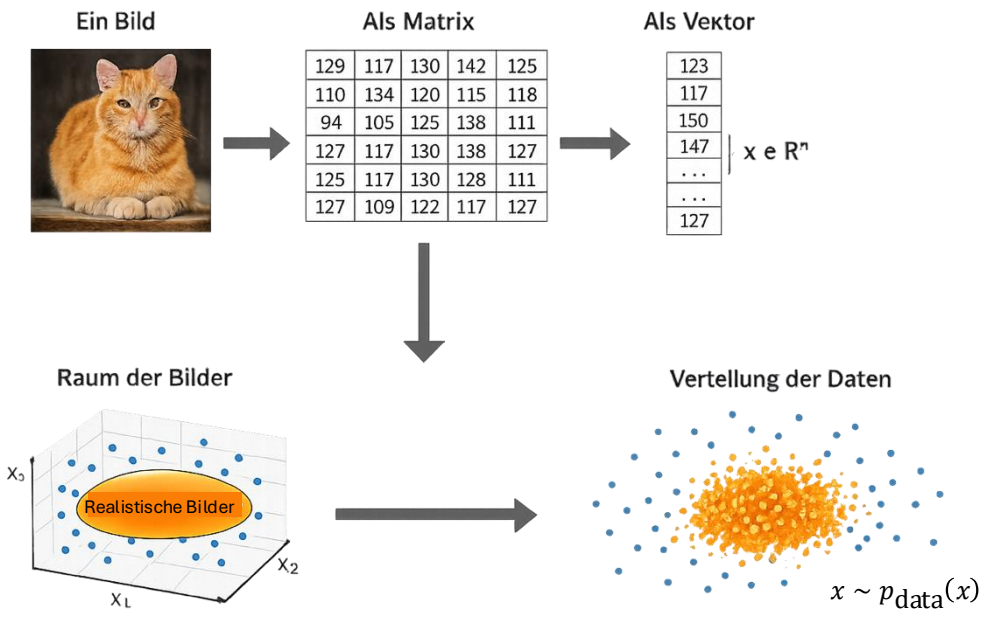


Mathematisch:

- $x \sim p_{\text{data}}(x)$

Interpretation:

- Ziel: Diese unbekannte Verteilung approximieren



Die Aufgabe generativer Modelle ist es, genau diese Verteilung zu lernen, also zu wissen, wo im Raum sich sinnvolle Bilder befinden

Agenda

- Was bedeutet „Generieren“ in der KI?
- Wie lassen sich Daten mathematisch beschreiben?
- **Wie lernen Modelle solche Verteilungen?**
- Beispiel: Generierung mit Diffusion Models
- Zusammenfassung

Generieren = Ziehen von Stichproben

Wenn wir $p(x)$ kennen, können wir:

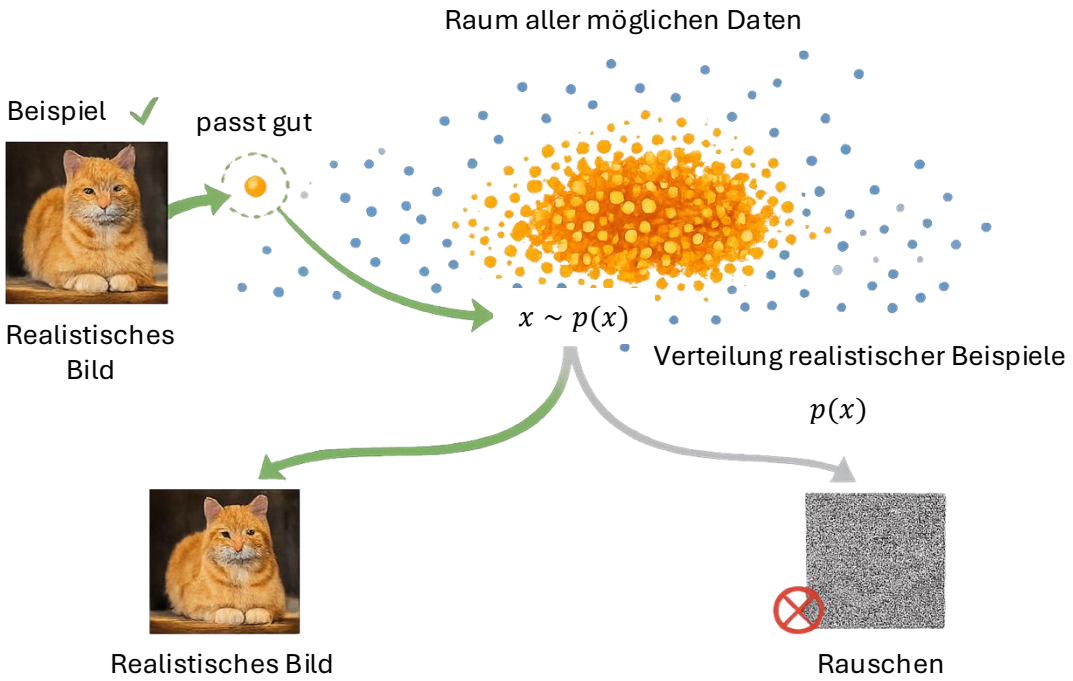
$$x \sim p(x)$$

→ neue Daten erzeugen

Das Modell erzeugt Bilder, die sich statistisch so verhalten wie echte Bilder

Beispiel:

- Bilder
- Texte
- Signale



- **Generative KI = Lernen + Sampeln**

Erzeugen neuer Beispiele

Das zentrale Problem

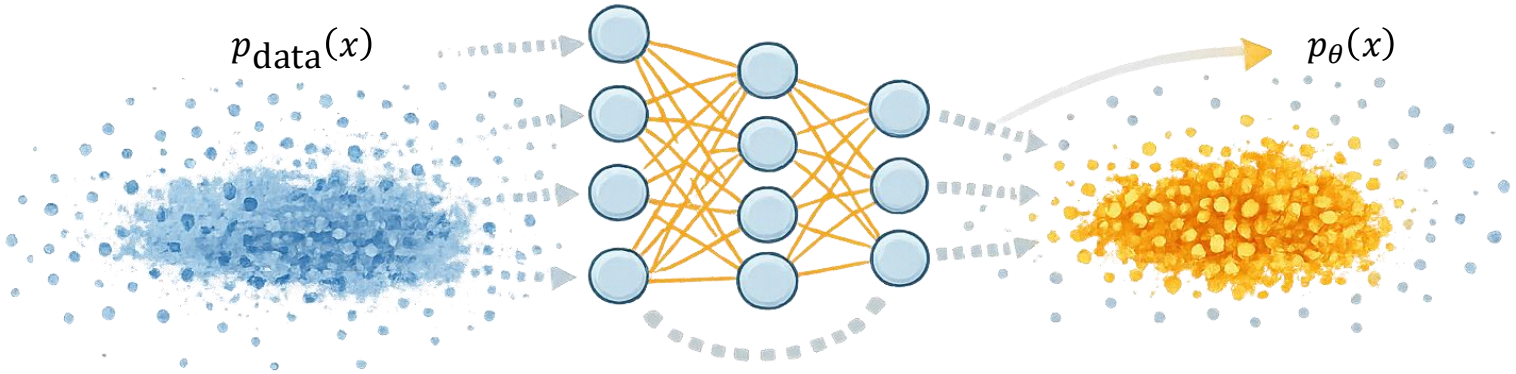
- Die echte Verteilung $p_{\text{data}}(x)$ ist unbekannt

Herausforderung:

- hochdimensional (z. B. Bilder)
- komplex
- nicht analytisch gegeben

Ziel:

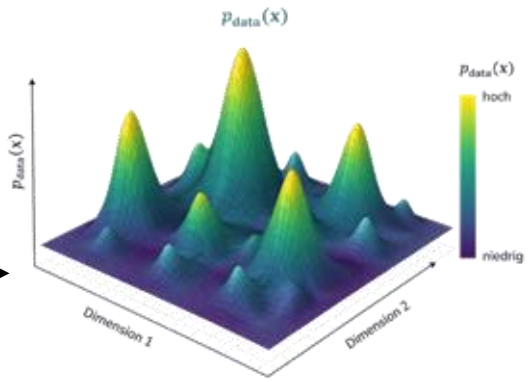
- $p_\theta(x) \approx p_{\text{data}}(x)$



Zentrale Idee generativer Modelle

Wir kennen bereits:

Echte Daten folgen einer Struktur (Verteilung $p_{\text{data}}(x)$)

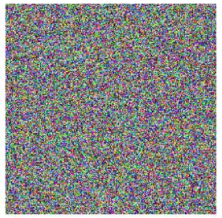


Unser Ziel:

Neue Daten x erzeugen, die zu dieser Struktur passen

Idee:

Wir starten mit etwas Einfachem (reiner Zufall z)



z.B. reines Rauschen

Und lernen eine Transformation f_θ, das daraus schrittweise sinnvolle Daten erzeugt

Zentrale Idee generativer Modelle

Wir starten mit einfacher Zufälligkeit → und transformieren sie in Daten

Schema:

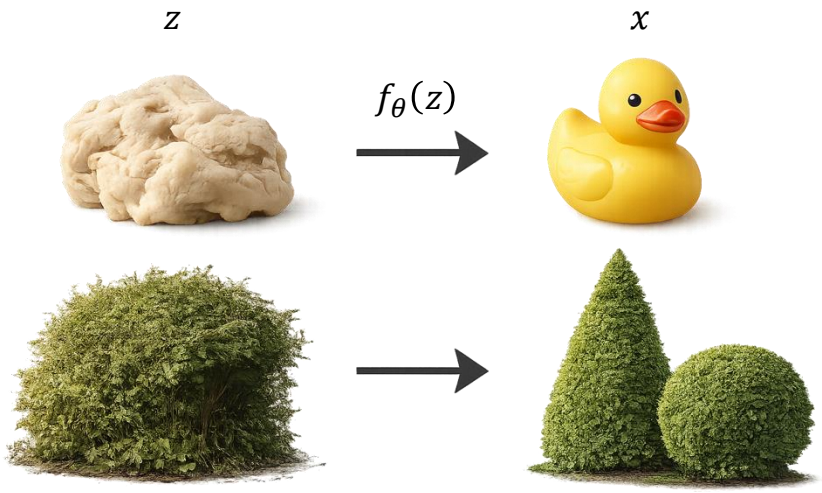
$$z \sim \mathcal{N}(0, I) \quad \Rightarrow \quad x = f_\theta(z)$$

Standard-Normalverteilung: Mittelwert 0, Varianz 1 (reines Rauschen)

Interpretation:

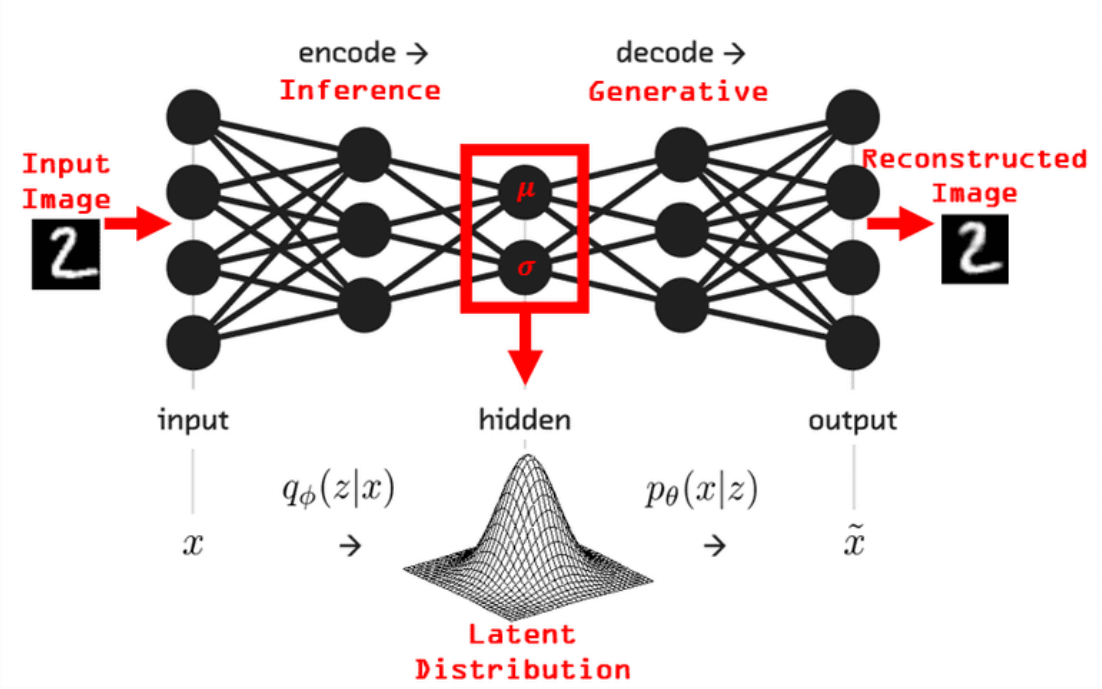
- z: einfacher Zufallsvektor
- f_θ: gelerntes Modell

→ Aus Rauschen wird Struktur



Vergleich generativer Ansätze

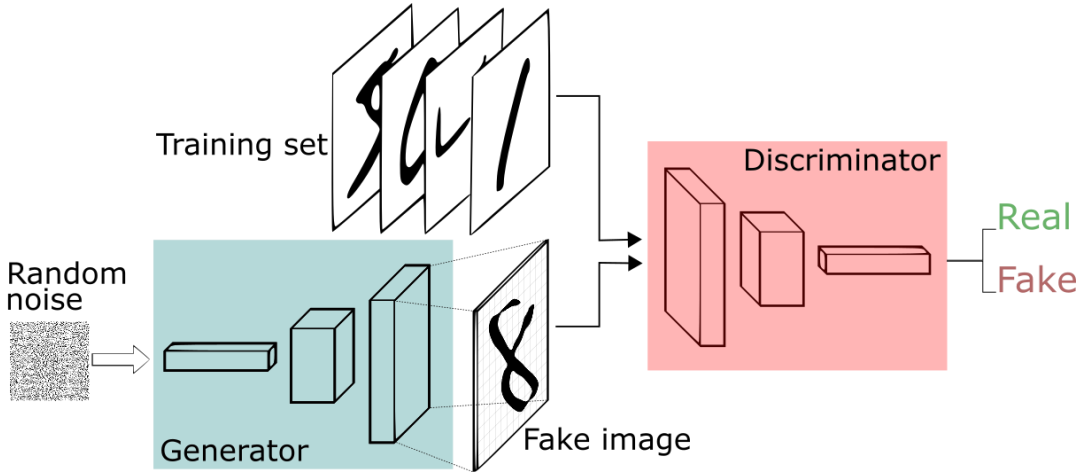
Modell	Idee	Herausforderung
VAE	**Latenter Raum → lernt eine komprimierte Darstellung der Daten und sampelt daraus neue Beispiele**	**oft unscharfe Ergebnisse**



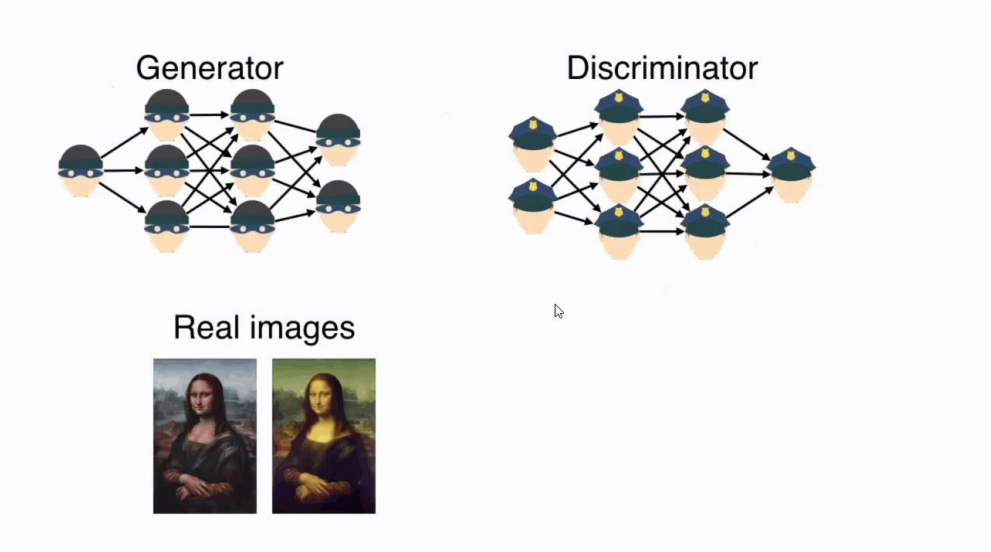
https://theaisummer.com/Autoencoder/

Vergleich generativer Ansätze

Modell	Idee	Herausforderung
VAE	Latenter Raum → lernt eine komprimierte Darstellung der Daten und sampelt daraus neue Beispiele	oft unscharfe Ergebnisse
GAN	**Wettbewerb → zwei Netzwerke (Generator & Diskriminator) konkurrieren miteinander**	**instabiles Training**



https://sthalles.github.io/intro-to-gans/



https://pub.towardsai.net/looking-back-at-the-original-vanilla-gan-and-its-evolution-over-time-3d441cedbd05

Vergleich generativer Ansätze

Modell	Idee	Herausforderung
VAE	Latenter Raum → lernt eine komprimierte Darstellung der Daten und sampelt daraus neue Beispiele	oft unscharfe Ergebnisse
GAN	Wettbewerb → zwei Netzwerke (Generator & Diskriminator) konkurrieren miteinander	instabiles Training
Autoregressive Modelle	**Faktorisierung → erzeugen Daten Schritt für Schritt (z. B. Pixel für Pixel oder Wort für Wort)**	**langsam bei der Generierung**

The cat

https://ai.plainenglish.io/next-word-prediction-enhancing-language-understanding-and-communication-1322f3b57632

Mathematisch: Berechnung der Wahrscheinlichkeit des nächsten Tokens w_i unter Einbeziehung des vorherigen (prior) Kontexts: $\boldsymbol{p(w_i|w_{<i})}$

$p(\text{Wort}_{t+1} \mid \text{alle vorherigen Wörter})$

https://theaisummer.com/Autoencoder/

Vergleich generativer Ansätze

Modell	Idee	Herausforderung
VAE	Latenter Raum → lernt eine komprimierte Darstellung der Daten und sampelt daraus neue Beispiele	oft unscharfe Ergebnisse
GAN	Wettbewerb → zwei Netzwerke (Generator & Diskriminator) konkurrieren miteinander	instabiles Training
Autoregressive Modelle	Faktorisierung → erzeugen Daten Schritt für Schritt (z. B. Pixel für Pixel oder Wort für Wort)	langsam bei der Generierung
Diffusion	**Rauschprozess**	**viele Schritte**

? → Kommt jetzt

Agenda

- Was bedeutet „Generieren“ in der KI?
- Wie lassen sich Daten mathematisch beschreiben?
- Wie lernen Modelle solche Verteilungen?
- **Beispiel: Generierung mit Diffusion Models**
- Zusammenfassung

Ein alternativer Ansatz: Diffusion Models

Idee geht zurück auf das Paper … (über 37k Zitierungen)

Eigentlich bestand die Idee schon im Jahr 2015 …

Bis dahin galten GANs als State-of-the-Art in der Bildgenerierung

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at https://github.com/hojonathanho/diffusion.

1 Introduction

Deep generative models of all kinds have recently exhibited high quality samples in a wide variety of data modalities. Generative adversarial networks (GANs), autoregressive models, flows, and variational autoencoders (VAEs) have synthesized striking image and audio samples [14, 27, 3, 58, 38, 25, 10, 32, 44, 57, 26, 33, 45], and there have been remarkable advances in energy-based modeling and score matching that have produced images comparable to those of GANs [11, 55].

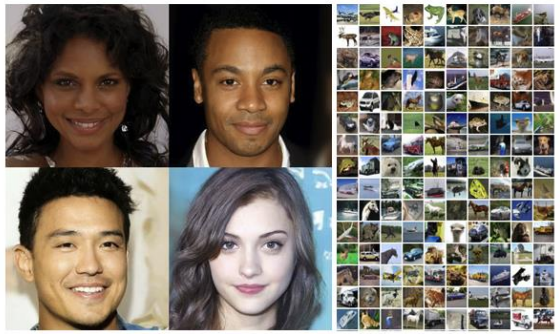


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

Denoising Diffusion Probabilistic Models @NeurIPS 2020

Deep Unsupervised Learning using Nonequilibrium Thermodynamics[1]

Jascha Sohl-Dickstein JASCHA@STANFORD.EDU
Stanford University

Eric A. Weiss EWEISS@BERKELEY.EDU
University of California, Berkeley

Niru Maheswaranathan NIRUM@STANFORD.EDU
Stanford University

Surya Ganguli SGANGULI@STANFORD.EDU
Stanford University

Abstract

A central problem in machine learning involves modeling complex data-sets using highly flexible families of probability distributions in which learning, sampling, inference, and evaluation are still analytically or computationally tractable. Here, we develop an approach that simultaneously achieves both flexibility and tractability. The essential idea, inspired by non-equilibrium statistical physics, is to systematically and slowly destroy structure in a data distribution through an iterative forward diffusion process. We then learn a reverse diffusion process that restores structure in data, yielding a highly flexible and tractable generative model of the data. This approach allows us to rapidly learn, sample from, and evaluate probabilities in deep generative models with thousands of layers or time steps, as well as to compute conditional and posterior probabilities under the learned model. We additionally release an open source reference implementation of the algorithm.

1. Introduction

Historically, probabilistic models suffer from a tradeoff between two conflicting objectives: *tractability* and *flexibility*. Models that are *tractable* can be analytically evaluated and easily fit to data (e.g. a Gaussian or Laplace). However, these models are unable to aptly describe structure in rich datasets. On the other hand, models that are *flexible* can be molded to fit structure in arbitrary data. For example, we can define models in terms of any (non-negative) function $\phi(\mathbf{x})$ yielding the flexible distribution $p(\mathbf{x}) = \frac{\phi(\mathbf{x})}{Z}$, where Z is a normalization constant. However, computing this normalization constant is generally intractable. Evaluating, training, or drawing samples from such flexible models typically requires a very expensive Monte Carlo process.

A variety of analytic approximations exist which ameliorate, but do not remove, this tradeoff–for instance mean field theory and its expansions (T, 1982; Tanaka, 1998), variational Bayes (Jordan et al., 1999), contrastive divergence (Welling & Hinton, 2002; Hinton, 2002), minimum probability flow (Sohl-Dickstein et al., 2011b;a), minimum KL contraction (Lyu, 2011), proper scoring rules (Gneiting & Raftery, 2007; Parry et al., 2012), score matching (Hyvärinen, 2005), pseudolikelihood (Besag, 1975), loopy belief propagation (Murphy et al., 1999), and many, many more. Non-parametric methods (Gershman & Blei, 2012) can also be very effective[1].

1.1. Diffusion probabilistic models

We present a novel way to define probabilistic models that allows:

1. extreme flexibility in model structure,
2. exact sampling,

Proceedings of the 32nd International Conference on Machine Learning, Lille, France, 2015. JMLR: W&CP volume 37.

Copyright 2015 by the author(s).

[1] Non-parametric methods can be seen as transitioning smoothly between tractable and flexible models. For instance, a non-parametric Gaussian mixture model will represent a small amount of data using a single Gaussian, but may represent infinite data as a mixture of an infinite number of Gaussians.

Deep Unsupervised Learning using Nonequilibrium Thermodynamics @PMLR 2015

Diffusion Models
Wo wird das eingesetzt?

Diffusionsmodelle sind generative KI-Modelle, die hochwertige Bilder, Videos oder Audiodaten aus zufälligem Rauschen erzeugen

Einsatz:

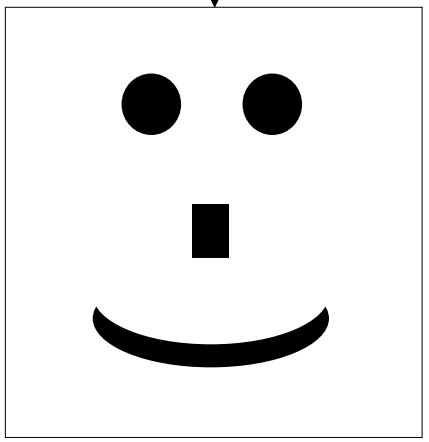
- Bildgenerierung (Stable Diffusion, DALL-E, Midjourney)
- Medizinische Bildgebung
- Datenaugmentation
- Simulation

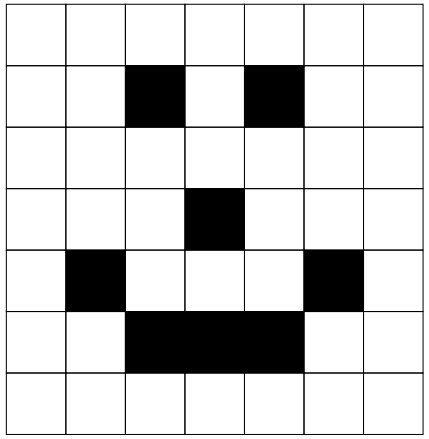
→ Große Bedeutung für Forschung & Praxis

Was sind Bilder?

Wir können das in einer Matrix x ausdrücken

Was wir sehen





255	255	255	255	255	255	255
255	255	0	255	0	255	255
255	255	255	255	255	255	255
255	255	255	0	255	255	255
255	0	255	255	255	0	255
255	255	0	0	0	255	255
255	255	255	255	255	255	255

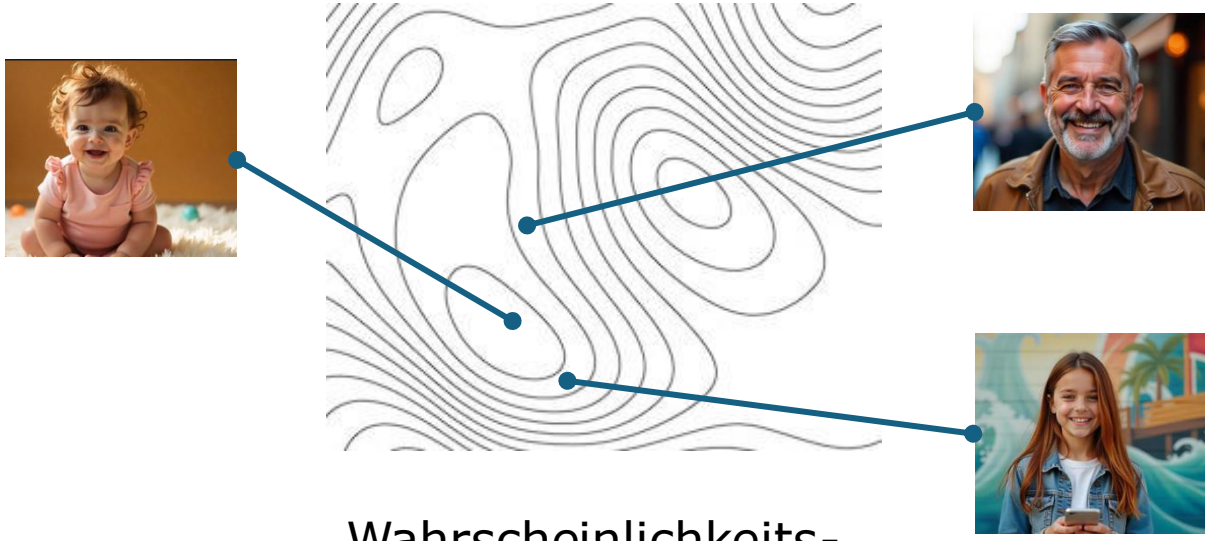
$$\Longrightarrow x \in \mathbb{R}^{7\times 7}$$

Was Computer sehen

Wahrscheinlichkeitsverteilung

$p(x)$ beschreibt eine Verteilung über alle möglichen Bilder und der Wert $p(x)$ sagt uns, wie typisch ein konkretes Bild ist

$$p(x)$$



Wahrscheinlichkeits-verteilung (/-bereich) von generierten Gesichtsbildern

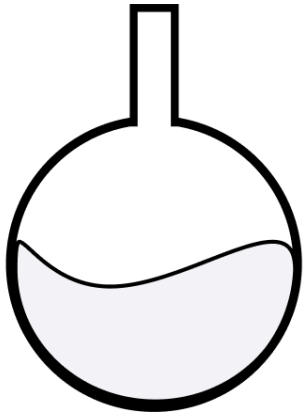
Die Verteilung ist so komplex, dass sie sich nicht mit einer einfachen Formel repräsentieren lässt

$$p(x) = ?$$

Wie wollen dennoch in der Lage sein, neue Bilder zu generieren

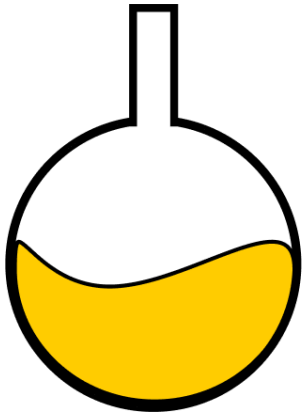
Diffusion Models Idee

Analogie aus der Physik (Thermodynamik)



Wasser

Hinzufügen einer kleinen Menge einer weiteren Flüssigkeit



Wasser ist nicht mehr transparent

https://towardsdatascience.com/diffusion-models-explained-simply/

Starten wir mit einem Bild

Jetzt übertragen auf ein Bild:

x_0

```
# ==================================================
# Diffusion-Forward
# ==================================================

def prepare_diffusion_constants(T, beta_start=1e-4, beta_end=0.02):
 """
 Erzeugt:
 - betas
 - alphas = 1 - beta_t
 - alpha_bars = Produkt der alphas bis t
 """
 betas = make_linear_beta_schedule(T, beta_start, beta_end)
 alphas = 1.0 - betas
 alpha_bars = np.cumprod(alphas)
 return betas, alphas, alpha_bars

def q_sample(x0, t, alpha_bars):
 """
 Direkte geschlossene Form des Forward-Prozesses:
 x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * epsilon
 wobei epsilon ~ N(0, I)
 Das ist die Standardformel des Forward Passes in Diffusion Models.
 """
 epsilon = np.random.randn(*x0.shape).astype(np.float32)
 alpha_bar_t = alpha_bars[t]
 x_t = np.sqrt(alpha_bar_t) * x0 + np.sqrt(1.0 - alpha_bar_t) * epsilon
 return x_t
```

Ein alternativer Ansatz: Diffusion Models

Wir geben ein paar Tropfen einer anderen Flüssigkeit hinzu bzw. in unserem Fall "wir fügen ein wenig Rauschen hinzu"

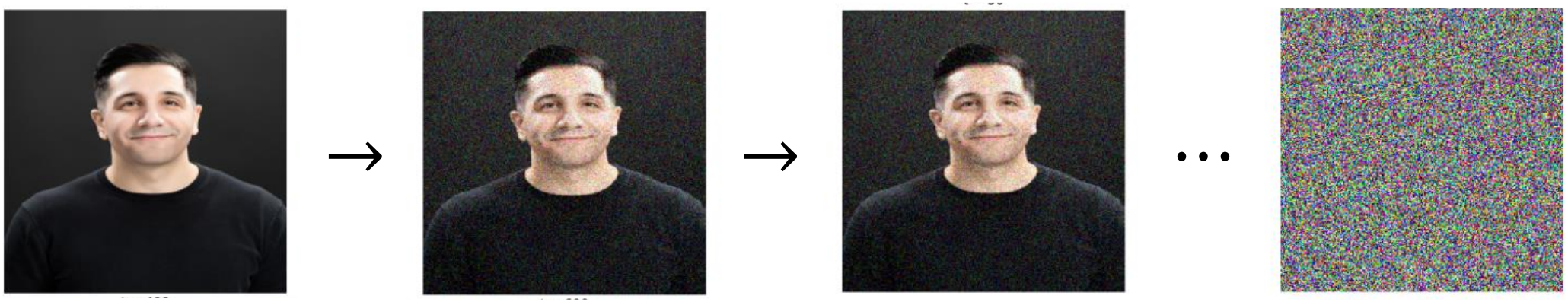
x_0

$\rightarrow$

x_1

Ein alternativer Ansatz: Diffusion Models

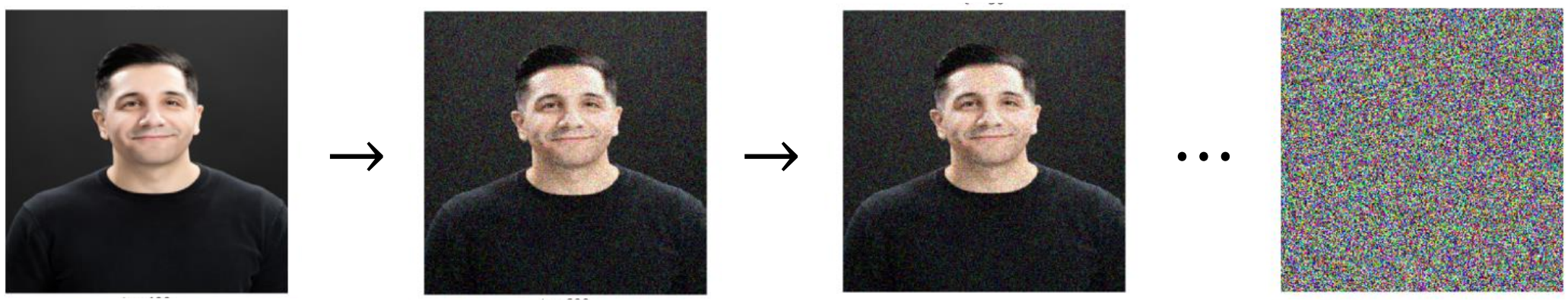
Nach T Iterationen (immer auf das vorherige Bild):



x_0 $\rightarrow$ x_1 $\rightarrow$ x_2 $\cdots$ x_T

Ein alternativer Ansatz: Diffusion Models

Rauschen über Normalverteilung (Gauß-Verteilung)



x_0 x_1 x_2 x_T

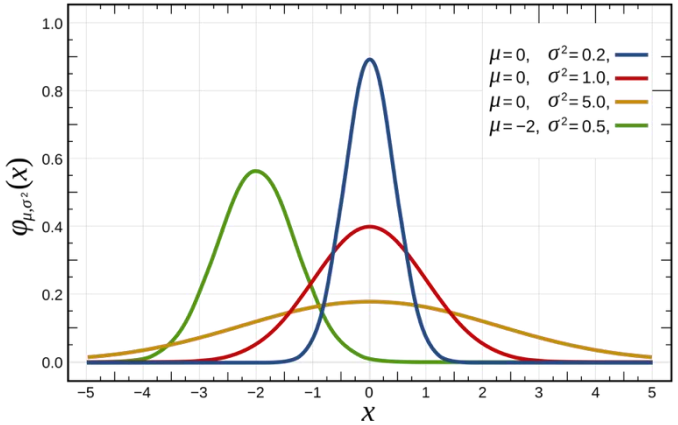
Ein alternativer Ansatz: Diffusion Models

Rauschen über Normalverteilung (Gauß-Verteilung)

Mittelwert μ und Standardabweichung σ

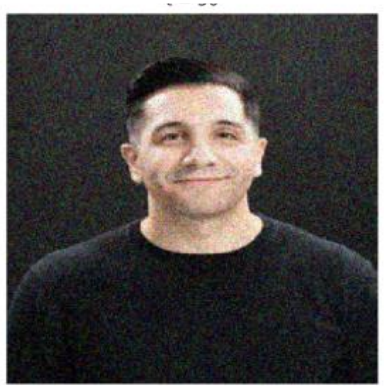
Dichtefunktionen der Normalverteilung $N(\mu, \sigma^2)$: $N(0; 0{,}2)$ (blau), $N(0; 1)$ (rot), $N(0; 5)$ (gelb) und $N(-2;, 0{,}5)$ (grün)

https://de.wikipedia.org/wiki/Normalverteilung

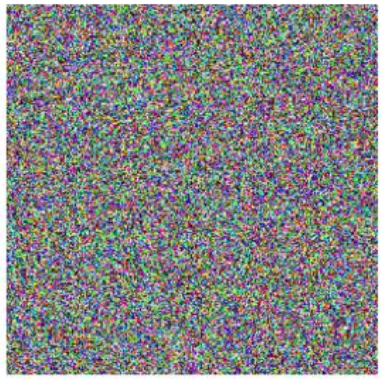


$\rightarrow$

$\rightarrow$



$\cdots$



x_0 x_1 x_2 x_T

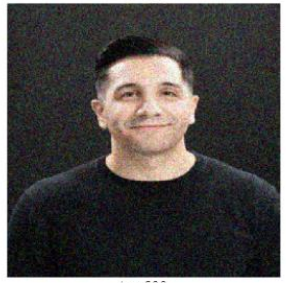
Ein alternativer Ansatz: Diffusion Models

D.h. unser Ausgangsbild setzt sich zusammen aus:

x_0

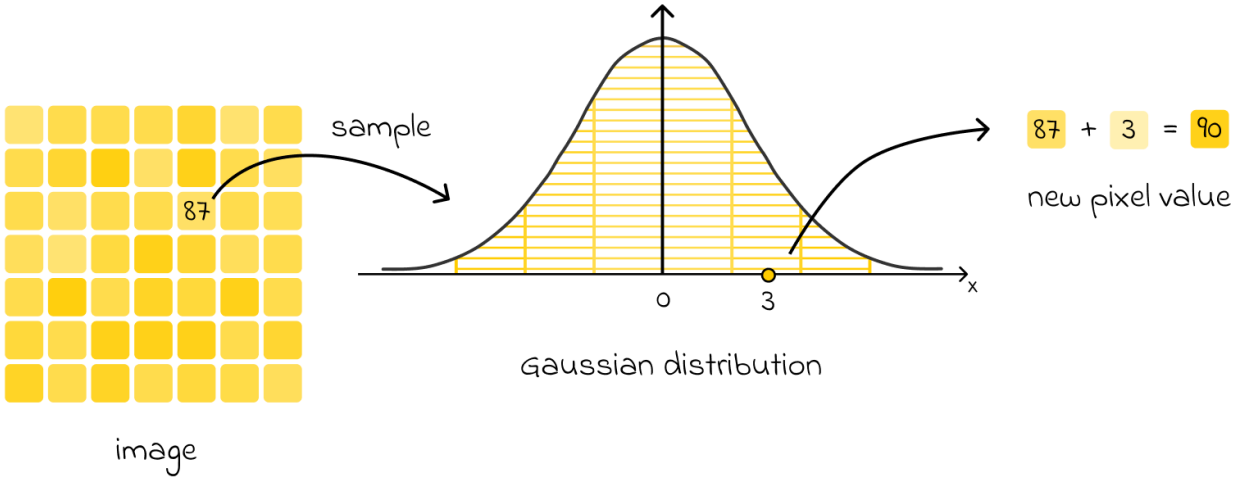
Original

→



x_1

etwas verrauscht



https://towardsdatascience.com/diffusion-models-explained-simply/

Ein alternativer Ansatz: Diffusion Models

Übertragen auf ein Bild lässt sich z.B. folgendes schreiben: $\mathcal{N}(0, I)$

Beispiel (ein Bild als Matrix):

$$x = \begin{bmatrix} 120 & 123 & 125 & 130 & 128 & 124 & 122 \\ 118 & 121 & 126 & 132 & 129 & 125 & 120 \\ 115 & 119 & 124 & 135 & 130 & 126 & 121 \\ 110 & 115 & 120 & 140 & 135 & 130 & 125 \\ 108 & 112 & 118 & 138 & 132 & 128 & 123 \\ 105 & 110 & 115 & 135 & 130 & 125 & 120 \\ 102 & 108 & 112 & 130 & 125 & 120 & 115 \end{bmatrix}$$

Zufälliges Rauschen ($\epsilon \sim \mathcal{N}(0, I)$):

$$\epsilon = \begin{bmatrix} 0.2 & -0.5 & 1.1 & 0.3 & -0.7 & 0.4 & -0.2 \\ -0.3 & 0.6 & -1.2 & 0.8 & 0.1 & -0.4 & 0.5 \\ 0.7 & -0.9 & 0.2 & -0.6 & 0.3 & 1.0 & -0.8 \\ -0.1 & 0.4 & -0.7 & 0.5 & -1.1 & 0.2 & 0.6 \\ 0.3 & -0.2 & 0.8 & -0.4 & 0.7 & -0.9 & 0.1 \\ -0.6 & 0.5 & -0.3 & 1.2 & -0.8 & 0.4 & -0.2 \\ 0.1 & -0.7 & 0.6 & -0.5 & 0.9 & -0.3 & 0.2 \end{bmatrix}$$

Erwartungswert:
$\mathbb{E}[\epsilon] = 0$

Varianz:
$\mathrm{Var}(\epsilon) = 1$

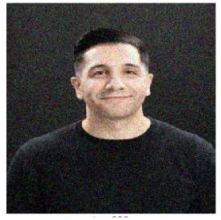
Angenommen die Matrix repräsentiert unser Bild

Zufälliges Rauschen

Wir können uns das wie folgt vorstellen:

x_0

$+ \rightarrow$

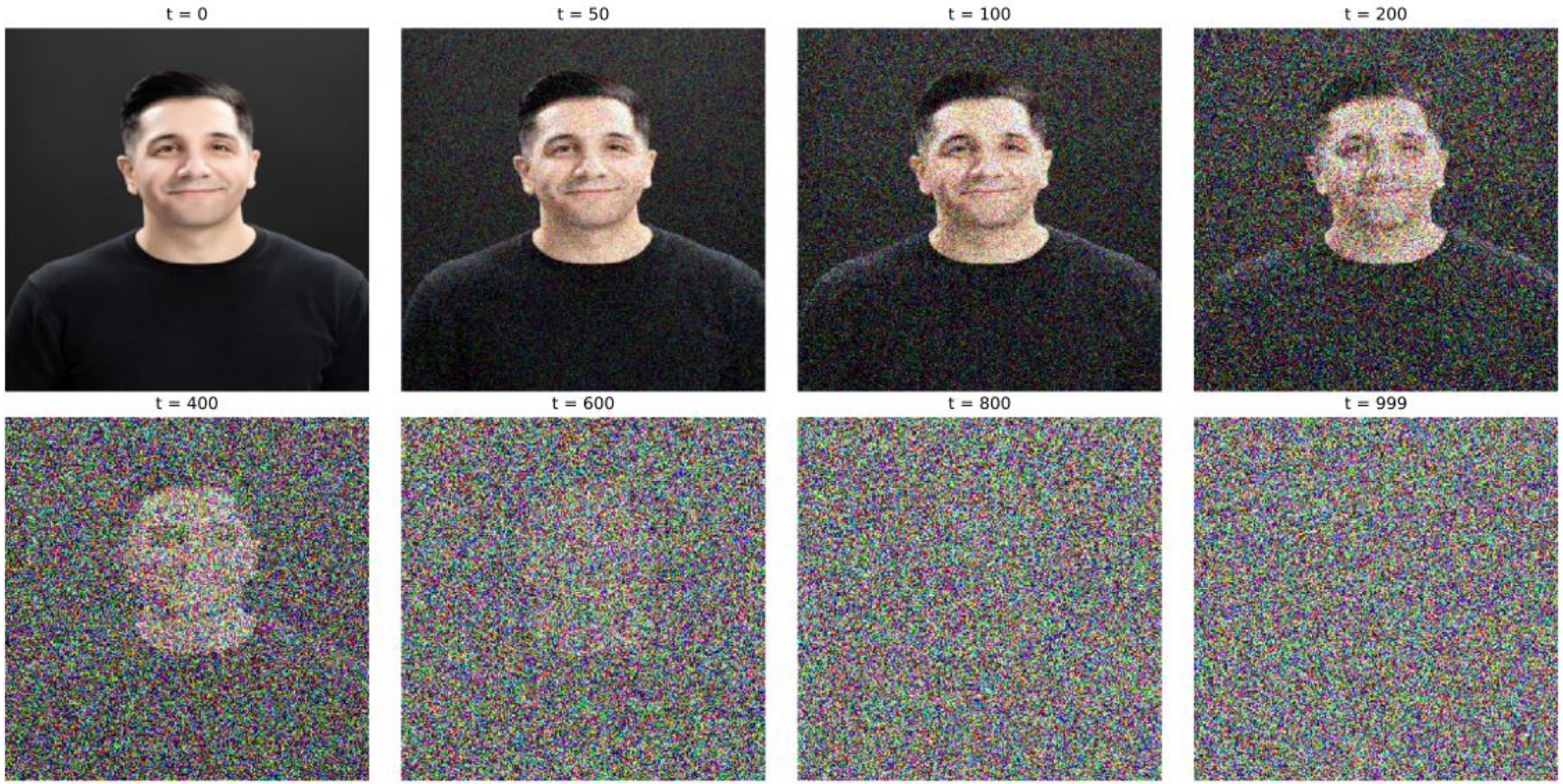


x_1

Intuition: Vom Bild zum Rauschen

- Schritt 1: Bild leicht verrauschen
- Schritt 2: mehr Rauschen
- …
- Am Ende: reines Rauschen

→ Prozess ist einfach modellierbar



Forward Process (Zerstörung)

Mathematisch:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \underbrace{\sqrt{1-\beta_t}\,x_{t-1}}_{\text{Mittelwert}}, \beta_t I\right)$$

Finales Bild oder auch Output → Rauschen

Vorgängerbild oder auch Input

Mittelwert

Erklärung:

- β_t: Rauschstärke
- Schrittweise wird Rauschen hinzugefügt

Hierbei handelt es sich um einen einzelnen Zeitschritt

→ Nach vielen Schritten:

$$x_T \sim \mathcal{N}(0, I)$$

Forward Process (Zerstörung)

Mathematisch:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right)$$

Wir können dies $\sqrt{1-\beta_t}$ auch abkürzen:

$$\alpha_t = \sqrt{1-\beta_t}$$

Das lässt sich in folgender Weise interpretieren:

- $\alpha_t = 0{,}9$ → 90% der Daten in dem Bild bleiben erhalten
- $\beta_t = 0{,}1$ → 10% Rauschen

Forward Process (Zerstörung)

Mathematisch:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right)$$

$$\alpha_t = \sqrt{1-\beta_t}$$

$$\overline{\alpha_t} = \prod_{s=1}^{t} \alpha_s$$

$$= \sqrt{1-\beta_t}.x_{t-1} + \sqrt{\beta_t}.\boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t}.x_{t-1} + \sqrt{1-\alpha_t}.\boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1}}x_{t-2} + \sqrt{1-\alpha_t \boldsymbol{\alpha}_{t-1}}\boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1} \boldsymbol{\alpha}_{t-2}}x_{t-3} + \sqrt{1-\alpha_t \boldsymbol{\alpha}_{t-1} \boldsymbol{\alpha}_{t-2}}\boldsymbol{\varepsilon}$$

Forward Process (Zerstörung)

Mathematisch:

$$\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

$$= \sqrt{1-\beta_t}.x_{t-1} + \sqrt{\beta_t}.\boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t}.x_{t-1} + \sqrt{1-\alpha_t}.\boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1}} x_{t-2} + \sqrt{1-\alpha_t \boldsymbol{\alpha}_{t-1}} \boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1} \boldsymbol{\alpha}_{t-2}} x_{t-3} + \sqrt{1-\alpha_t \boldsymbol{\alpha}_{t-1} \boldsymbol{\alpha}_{t-2}} \boldsymbol{\varepsilon}$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1} \cdots . \boldsymbol{\alpha}_1 \boldsymbol{\alpha}_0} x_0 + \sqrt{1-\alpha_t \boldsymbol{\alpha}_{t-1} \cdots . \boldsymbol{\alpha}_1 \boldsymbol{\alpha}_0} \boldsymbol{\varepsilon}$$

Forward Process (Zerstörung)

Mathematisch:

$$\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

$$= \sqrt{\alpha_t \boldsymbol{\alpha}_{t-1} \cdots . \boldsymbol{\alpha}_1 \boldsymbol{\alpha}_0} x_0 + \sqrt{1 - \alpha_t \boldsymbol{\alpha}_{t-1} \cdots . \boldsymbol{\alpha}_1 \boldsymbol{\alpha}_0} \boldsymbol{\varepsilon}$$

$$= \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t} \boldsymbol{\varepsilon}$$

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\overline{\alpha}_t} x_0, (1 - \overline{\alpha}_t) I\right)$$

Originalbild

Verrauschtes Bild

Frage: Was jetzt?

Was habe ich jetzt erreicht und wie kann ich von hier weitermachen?

Rückwärtsdenken

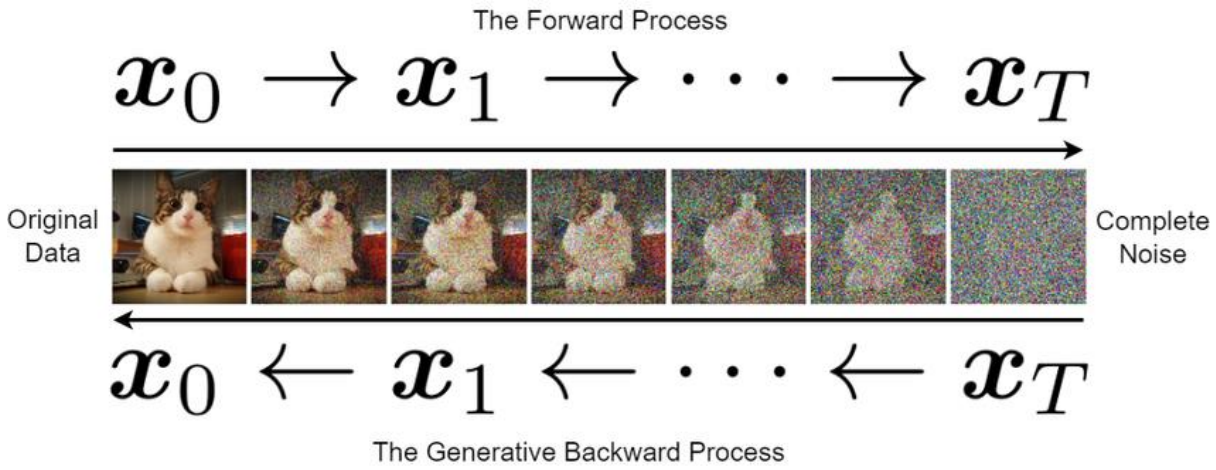
Statt direkt zu generieren:

Idee:

- Daten → schrittweise zerstören
- Modell lernt: diesen Prozess umzukehren

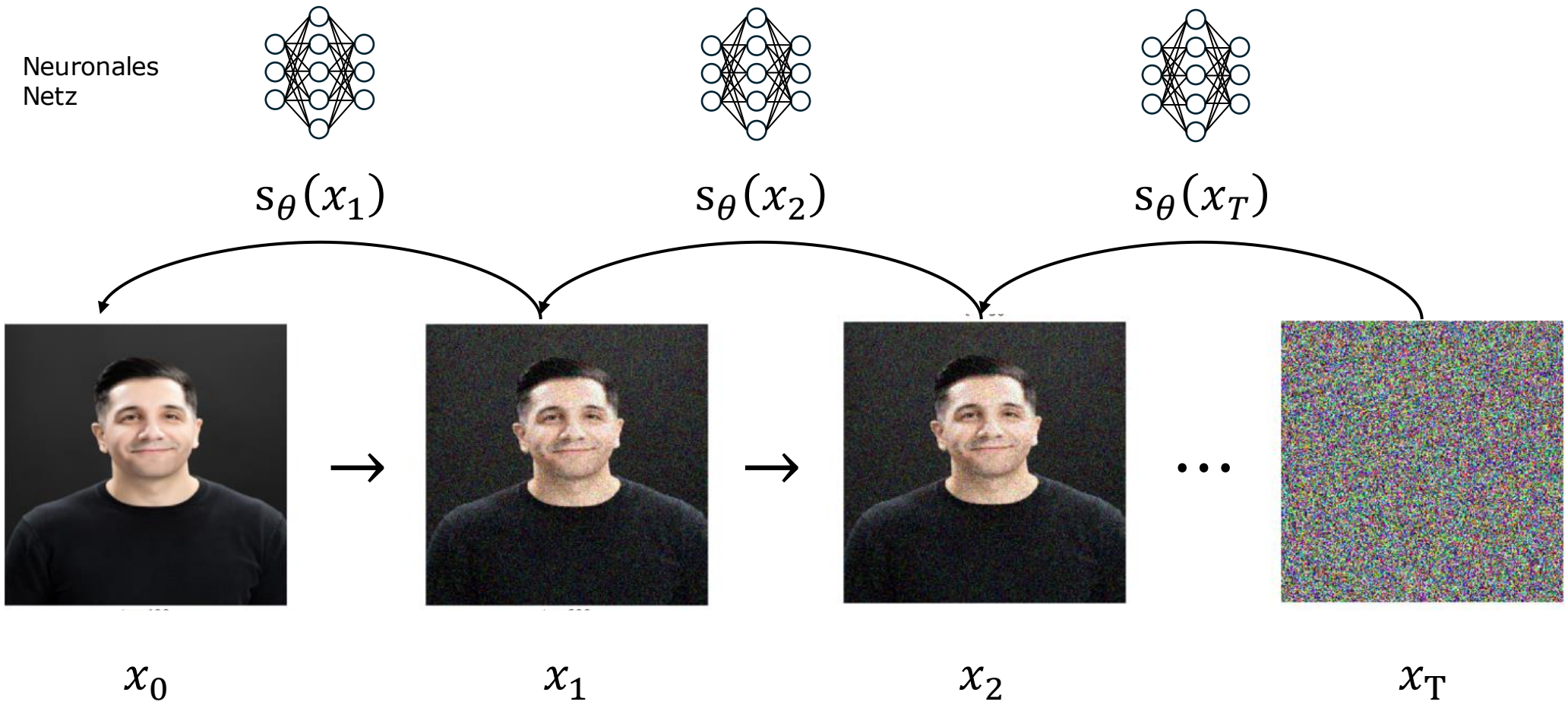
Kernaussage:

- Lernen durch „Rückwärtsdenken“



researchgate.net/publication/382128283_Diffusion_Models_Tutorial_and_Survey/figures?lo=1&utm_source=google&utm_medium=organic

Ein alternativer Ansatz: Diffusion Models



Reverse Process (Generierung)

Ziel: Rauschen wieder entfernen

$$p(\, x_{t-1} \mid x_t \,)$$

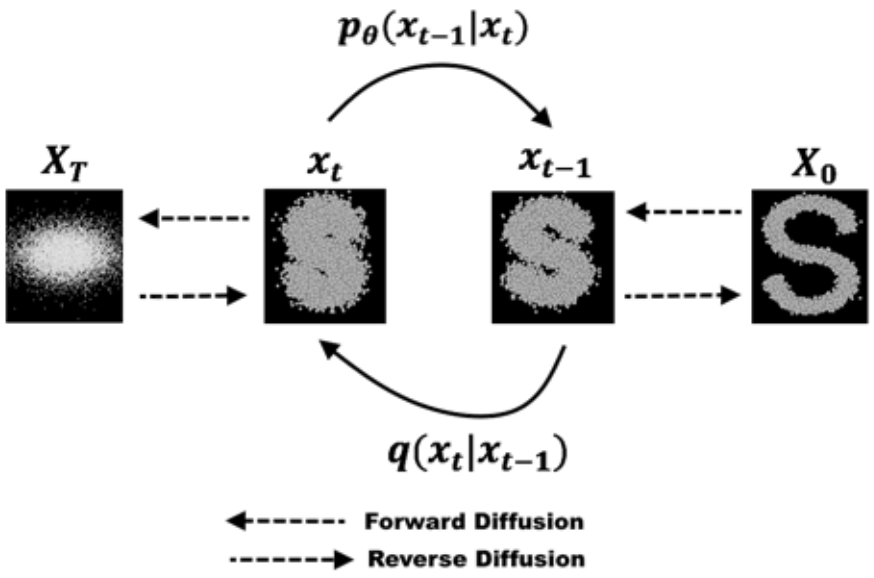
Weniger verrauschtes Bild

Verrauschtes Bild

Modell lernt:

- Wie sah der vorherige Zustand aus?

→ Schritt für Schritt entsteht ein Bild



https://towardsdatascience.com/diffusion-models-made-easy-8414298ce4da/

Reverse Process (Generierung)

Ziel: Rauschen wieder entfernen

$$p_\theta(x_{t-1} \mid x_t)$$

Weniger verrauschtes Bild

Verrauschtes Bild

Das Modell (θ) schätzt Mittelwert und Unsicherheit, um aus x_t den vorherigen Zustand x_{t-1} zu rekonstruieren

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\big)$$

Output Bild

Erwartungswert

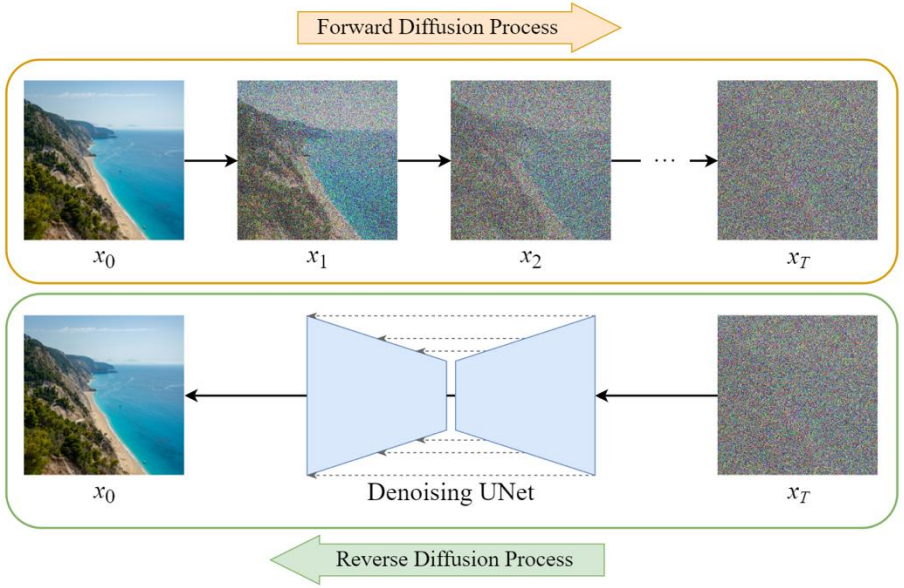
Varianz

Vorhersage

Modell lernt:

Wie sah der vorherige Zustand aus?

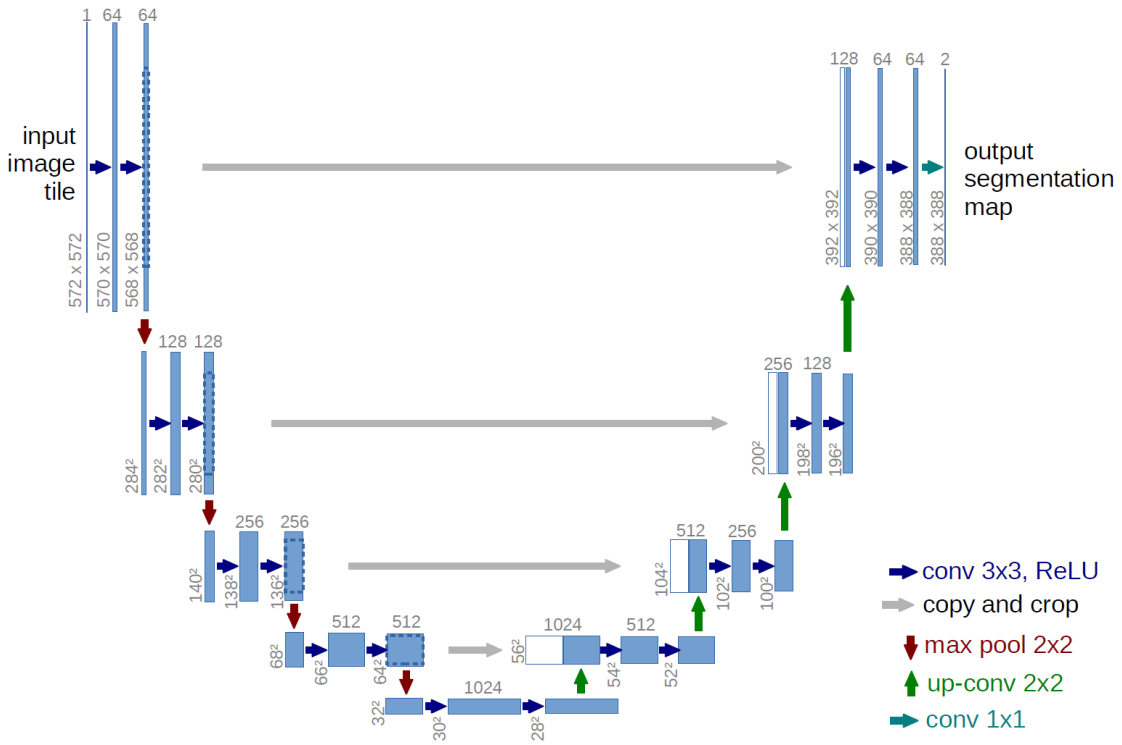
→ Schritt für Schritt entsteht ein Bild



https://medium.com/@steinsfu/diffusion-model-clearly-explained-cd331bd41166

Encoder – Decoder – Skip Connections

- Entwickelt 2015 für Bio-Bildsegmentierung (Ronneberger et al.)
- **Symmetrisches Encoder-Decoder-Netzwerk**
- Ziel: **Lernen von Kontexteigenschaften (Encoder) + Präzise Lokalisierung (Decoder)**
- **Encoder**: Convolution + ReLU + MaxPooling (Feature-Extraktion, Kontext)
- **Decoder**: UpConvolution (Transposed Conv) + Convolution (Rekonstruktion)
- **Skip Connections**: Kopieren Feature Maps vom Encoder → Decoder
  → Erhalten feinräumige Details, verbessern Lokalisierung
- Charakteristisches U-förmiges Layout → Name „U-Net“



https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/

Was wird eigentlich gelernt?

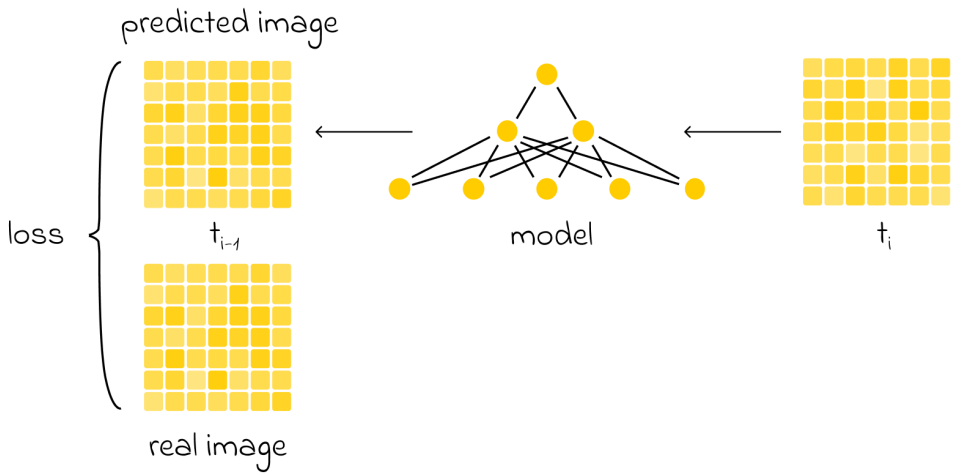
Modell lernt, das Rauschen zu schätzen

$$\epsilon_\theta(x_t, t)$$

Interpretation:

- Vorhersage des hinzugefügten Rauschens
- Subtraktion → sauberes Signal

→ Einfaches Lernziel, starke Wirkung



https://towardsdatascience.com/diffusion-models-explained-simply/

Trainingsziel

Mathematisch:

$$\mathcal{L} = \mathbb{E}[|\epsilon - \epsilon_\theta(x_t, t)|^2]$$

Wahres Rauschen (aus dem Forward Prozess)

Vorhersage des Rauschens

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

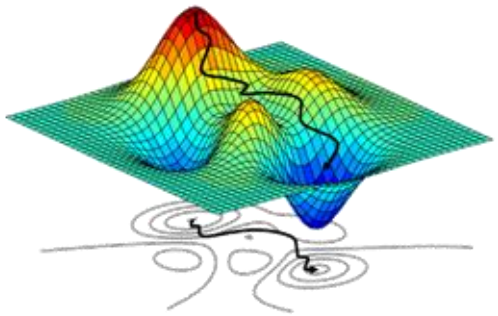
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Interpretation:

- Modell soll echtes Rauschen rekonstruieren

→ MSE als Loss

→ Ziel: Loss minimieren

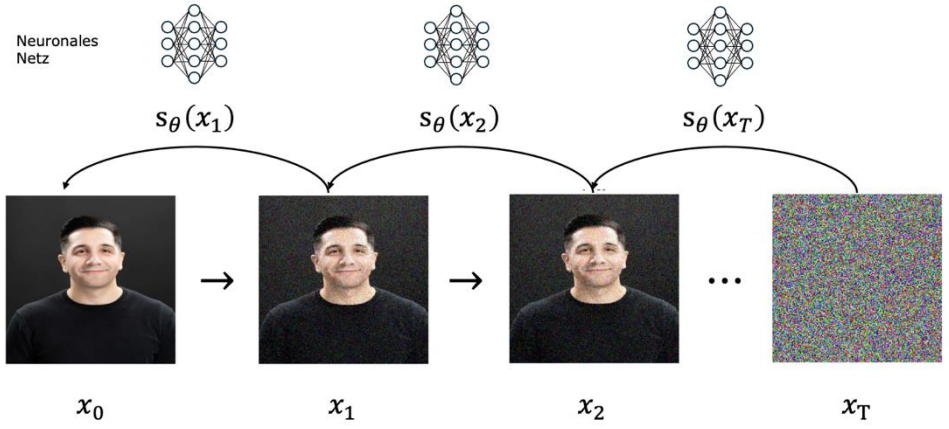


Agenda

- Was bedeutet „Generieren“ in der KI?
- Wie lassen sich Daten mathematisch beschreiben?
- Wie lernen Modelle solche Verteilungen?
- Beispiel: Generierung mit Diffusion Models
- **Zusammenfassung**

Zusammenfassung

- Mathematik ist die Grundlage moderner KI-Systeme
- Generative Modelle lernen Verteilungen
- Ziel: neue Daten erzeugen
- Diffusion = aktueller State-of-the-Art Ansatz:
  - Vorwärts: Rauschen hinzufügen
  - Rückwärts: Struktur rekonstruieren



→ Lernen durch Invertieren eines Prozesses

Weiterführende Quellen

Videos

- Diffusion Models: DDPM | Generative AI Animated (Deepia ~ 32 min)
- But how do AI images and videos actually work? | Guest video by Welch Labs (3Blue1Brown and Welch Labs ~ 37 min)
- Diffusion Models | Paper Explanation | Math Explained (Outlier ~ 33 min)

Folien

- Diffusion (Hao Chen @Carnegie Mellon University)
- Variational Autoencoders and Diffusion Models (Ruiqi Gao @Stanford cs231n)

Bücher

- Hands-On Generative AI with Transformers and Diffusion Models (Omar Sanseviero et al.)
- Learn Generative AI with PyTorch (Mark Liu)

Kontakt

Mail:
konak@uni-potsdam.de

Folien von heute

https://github.com/konakion/genai